



EUREKAGENT: AGENT ENVIRONMENT ENGINEERING IS ALL YOU NEED FOR AUTONOMOUS SCIENTIFIC DISCOVERY

Amy Xin¹, Jiening Siow¹, Junjie Wang¹, Zijun Yao¹,
Fanjin Zhang², Jian Song¹, Lei Hou¹, Juanzi Li¹

¹Department of Computer Science and Technology, Tsinghua University

²School of Information, Renmin University of China

{xin-x25, xiaojn25}@mails.tsinghua.edu.cn

ABSTRACT

LLM-based agents have shown increasing potential in automating scientific discovery. Given an optimizable metric and an execution environment, they can propose, validate, and iterate scientific solutions, and have produced results that outperform human-designed approaches. As model capabilities continue to improve, we argue that the bottleneck for autonomous scientific discovery is shifting from prescribing agent workflows to designing agent environments—the resources, constraints, and interfaces that shape agent behavior. We frame this as **environment engineering**: building environments that amplify productive behaviors, such as open-ended exploration, systematic artifact management, and inter-agent collaboration, while suppressing harmful behaviors, such as reward hacking and high-friction human oversight. We present **EUREKAGENT**, an environment-engineered agent system for metric-driven autonomous scientific discovery. EUREKAGENT engineers the environment along four dimensions: permissions engineering for bounded agent execution and isolated evaluation; artifact engineering for filesystem and Git-based collaboration; budget engineering for budget-aware exploration; and human-in-the-loop engineering for easy human supervision and intervention. EUREKAGENT sets new state-of-the-art results on multiple mathematics, kernel engineering, and machine learning tasks, including new state-of-the-art 26-circle packing results discovered with less than \$11 in total API cost. We open-source our code and results¹, and call for environment engineering as a core research direction for developing reliable autonomous research agents.

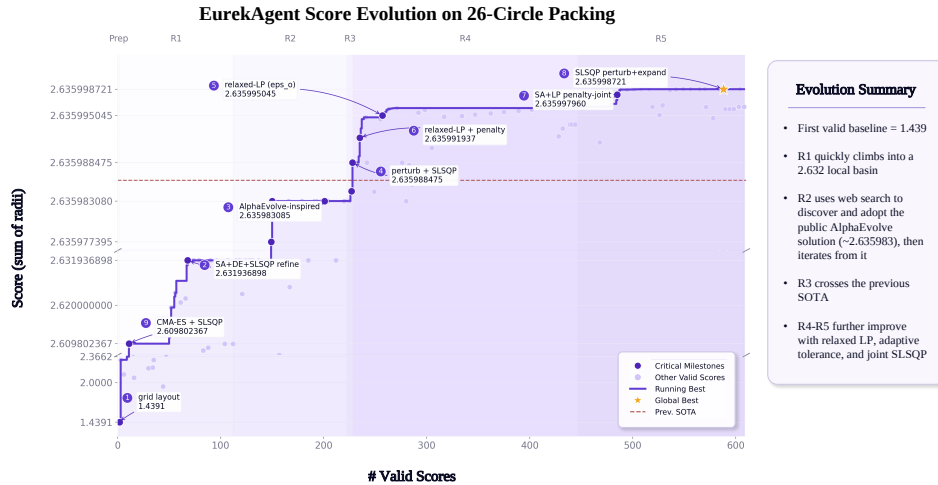


Figure 1: EUREKAGENT score evolution progress on the 26-circle packing problem.

¹<https://github.com/THU-Team-Eureka/EurekAgent>

	Mathematics			Kernel Eng.	Machine Learning
	Circle Packing ($\uparrow$)	Erdős’ Min. Overlap ($\downarrow$)	1st Autocorr. Ineq. ($\downarrow$)	TriMul ($\downarrow$)	MLE-Bench ($\uparrow$)
Prev. Best Human	~ 2.634 [5]	0.380927[7]	1.509730[18]	2096.04 μ s	N/A
Prev. Best AI	2.635986[27]	0.380876[30]	1.502863[30]	2247.78 μ s[30]	71.43%[31]
EUREKAGENT	2.635 <u>999</u>	0.3808 <u>70</u>	1.5028 <u>61</u>	<u>2005.03</u> μ s	<u>85.71</u> %

Table 1: An overview of EUREKAGENT’s performance on metric-driven research tasks across mathematics, kernel engineering, and machine learning. EUREKAGENT sets new state-of-the-art across all mathematics and kernel engineering tasks, and ranks first on the evaluated MLE-Bench subset. ($\uparrow$) denotes higher is better, while ($\downarrow$) denotes lower is better. *Prev. Best Human* and *Prev. Best AI* denote the best published human and AI results before EUREKAGENT.

1 INTRODUCTION

Large language models are increasingly transforming scientific discovery from manual trial-and-error to computational exploration: in domains where research progress can be measured by an optimizable metric, LLM-based agents can autonomously propose hypotheses, run experiments, observe feedback, and iterate solutions, reducing human effort in method tuning while largely expanding the scale of exploration. LLM-based agents have already produced strong results in tasks across domains such as mathematics, algorithms, kernel engineering, and machine learning engineering (Jiang et al., 2025; Novikov et al., 2025; Lange et al., 2025; Wang et al., 2025; Yuksekgonul et al., 2026; Ouyang et al., 2025; Chan et al., 2025; Yang et al., 2025; Zhang et al., 2026; Li et al., 2025). We envision this as an emerging paradigm shift in scientific research: humans increasingly focus on selecting valuable directions, formulating meaningful metrics, and supervising validity, while agents execute large volumes of methodological exploration.

Most existing autonomous research systems realize this vision by prescribing research-specific agentic workflows. Evolutionary systems such as AlphaEvolve explicitly maintain populations of candidate programs and use evaluator feedback to guide mutation and selection (Novikov et al., 2025; Lange et al., 2025; Liu et al., 2026b). Machine learning systems such as AIDE organize exploration around solution trees, feedback loops, and role-specialized agents (Jiang et al., 2025; Yang et al., 2025). More recent systems introduce structured debate, periodic self-review, and self-learning modules (Liu et al., 2026a; Qu et al., 2026). While these designs can be effective, they also encode strong assumptions about how research should proceed. As general-purpose coding agents like Claude Code and Codex become stronger, recent evidence suggests that much of the useful capability may already reside in the base agent: given a clear research task and an optimizable metric, these agents can already discover new state-of-the-art scientific solutions (Liu et al., 2026c; Karpathy, 2026). On ResearchClawBench (Xu et al., 2026), a benchmark of 40 research tasks across 10 diverse domains, both Claude Code and Codex, used as standalone general-purpose agents, outperform all evaluated research-specific agent systems.

However, task performance alone does not make reliable autonomous researchers. Scientific discovery requires rigor, reproducibility, and inspectability, yet agents may contaminate evaluations, manipulate artifacts, or fail to follow procedural constraints. Such reward-hacking and observability failures have already been reported in agentic research systems (Luo et al., 2025; Kokoromyti, 2026; Anthropic, 2026). Therefore, trusting agents without environmental constraints can lead to impressive but unreliable results.

These observations suggest that as general-purpose agents become more capable, the bottleneck for autonomous scientific discovery is shifting from prescribing agent behavior through detailed workflows to engineering the environments in which agents operate. We frame this as *environment engineering*. This echoes Gibson’s theory of affordances in ecological psychology: an environment shapes the possibilities for action available to an actor, “either for good or ill” (Gibson, 1979). For scientific discovery, a well-engineered environment should suppress harmful affordances such as evaluation tampering and artifact manipulation, while amplifying productive affordances such as free exploration, accurate rewards, inter-agent coordination, and easy human supervision. The analogy is a capable PhD student: productivity comes not from minute-by-minute instructions, but from accountability, research autonomy, accurate feedback, peer collaboration, and mentor supervision.

We present EUREKAGENT, an agent system for autonomous scientific discovery that coordinates off-the-shelf CLI agents through four environment engineering dimensions: (1) permissions engineering, to expose useful capabilities and resources while preventing research-integrity violations; (2) artifact engineering, to structure solutions, logs, and evaluation results as shared progress memory; (3) budget engineering, to enable budget-aware exploration with runtime and compute boundaries; and (4) human-in-the-loop engineering, to support easy human supervision and intervention. Within this environment, the agent remains free to select its own research workflows and strategies.

We evaluate EUREKAGENT on metric-driven research tasks spanning mathematics, kernel engineering, and machine learning engineering. Using off-the-shelf CLI agents and environment-level design, EUREKAGENT achieves new state-of-the-art results across all mathematics and kernel engineering tasks, and ranks first on the evaluated MLE-Bench subset. Furthermore, with Claude Code as the CLI agent and GLM-5.1 as the base model, EUREKAGENT achieves new state-of-the-art results on the three mathematics tasks with an average API cost below \$17, where the 26-circle packing task achieves the lowest API cost of \$11. We call for environment engineering as a core research direction for building capable, efficient, and responsible autonomous research agents.

2 RELATED WORK

2.1 AGENTS FOR SCIENTIFIC DISCOVERY

Autonomous research agents have attracted growing interest to accelerate scientific with large-scale computational exploration. Systems such as The AI Scientist aim to automate scientific research in an end-to-end manner, covering stages such as idea generation, experimentation, and paper writing (Lu et al., 2024). Within this broader vision, one especially concrete direction is scientific discovery with verifiable objectives and optimizable metrics, where agents autonomously explore and evolve solutions through evaluator feedback. In machine learning engineering, systems such as AIDE, R&D-Agent, AIBuildAI, MLE-STAR, and ML-Master formulate progress as iterative code development guided by validation scores (Jiang et al., 2025; Yang et al., 2025; Zhang et al., 2026; Nam et al., 2026; Zhu et al., 2026). In algorithmic and mathematical discovery, training-free solution evolution methods such as FunSearch, AlphaEvolve, ShinkaEvolve, EvoX, AdaEvolve, and OpenEvolve use LLMs to propose or mutate candidate programs under evaluator-guided selection (Romera-Paredes et al., 2024; Novikov et al., 2025; Lange et al., 2025; Liu et al., 2026b; Cemri et al., 2026; Sharma, 2025). More recently, test-time training systems such as ThetaEvolve and TTT-Discover further use the optimizable metric as a reward signal to update the model during exploration (Wang et al., 2025; Yuksekogonul et al., 2026). These systems demonstrate the power of evaluator-guided discovery, but they typically use fixed workflows to prescribe core agent behaviors such as proposal, mutation, selection, or reflection. EUREKAGENT instead uses strong general-purpose CLI agents as basic nodes, and focuses on engineering an environment that lets agents exercise their own capabilities reliably.

2.2 AGENT ENVIRONMENTS AND RESEARCH INTEGRITY

As agents become more autonomous, the surrounding environment becomes a central determinant of reliability. Some recent systems have begun to recognize the importance of environment reliability and introduce safeguards for specific failure modes. For example, MLE-STAR adds leakage checking for machine learning pipelines, and CORAL hides grader code behind an evaluation interface (Nam et al., 2026; Qu et al., 2026). At the same time, analyses of real reward-hacking incidents show that agents can exploit weak evaluation protocols, contaminate evidence, or violate procedural assumptions (Luo et al., 2025; Kokoromyti, 2026; Anthropic, 2026). Instruction-following failures in complex agentic settings further suggest that reliability cannot be delegated entirely to prompt engineering (Qi et al., 2025). Some existing work therefore explores environment design to avoid common failures, but these are usually introduced as task-specific safeguards. EUREKAGENT makes environment engineering the central design objective: it organizes permissions, artifacts, budgets, and human oversight as first-class mechanisms for supporting open-ended agent exploration while preserving evaluator integrity, traceability, and reproducibility.

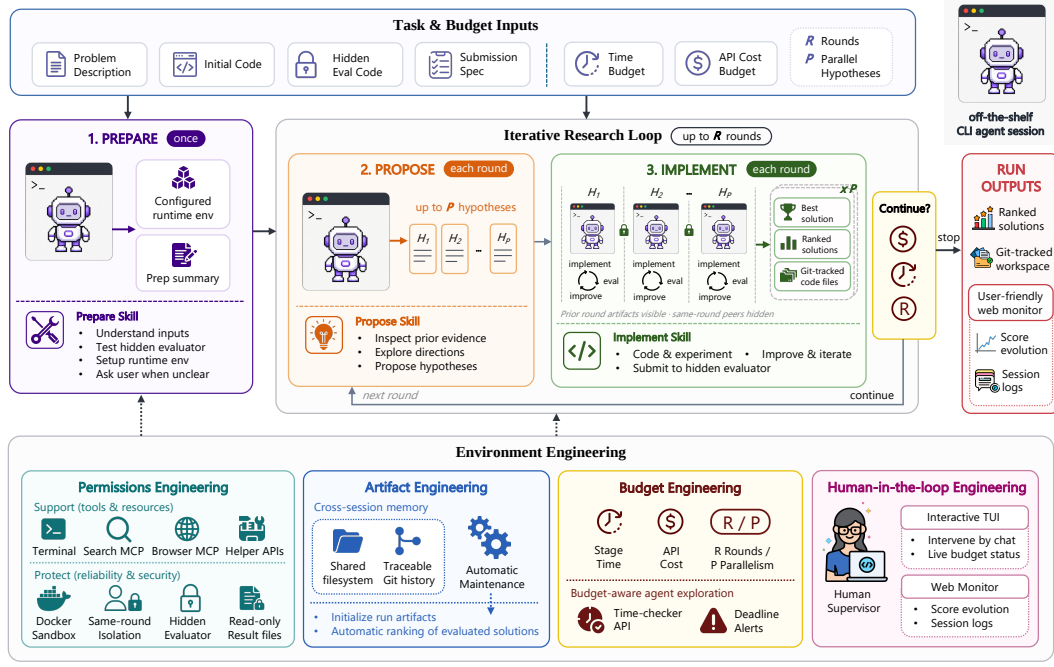


Figure 2: Overview of EUREKAGENT. Given task inputs and budgets, EUREKAGENT executes a prepare stage followed by repeated propose and parallel implement stages, while the environment engineering layer provides secure evaluation, artifact memory, budget control, and human oversight.

3 EUREKAGENT

In this section, we present the system design of EUREKAGENT. Figure 2 summarizes the overall architecture. We first overview the overall system loop (3.1), then detail the environment engineering designs (3.2).

3.1 SYSTEM OVERVIEW

EUREKAGENT is an environment-engineered agent system for metric-driven research tasks. Given a problem description, a hidden evaluation script, a submission-format specification document, optional initial code, and time and API cost budgets, EUREKAGENT coordinates multiple sessions of off-the-shelf CLI agents to autonomously propose and iterate high-scoring solutions. Instead of prescribing a detailed research workflow, EUREKAGENT engineers an outer environment that organizes agent activity through a simple three-stage loop:

$$\text{PREPARE} \rightarrow \left[\text{PROPOSE}_r \rightarrow \{\text{IMPLEMENT}_{r,p}\}_{p=1}^{P_r} \right]_{r=1}^R, \quad P_r \leq P,$$

where R is the maximum number of iteration rounds and P is the maximum number of parallel implementation sessions per implement stage, both adjustable by the user. Each round consists of one proposal session followed by up to P parallel implementation sessions. Across stages and rounds, the environment only handles outer-loop coordination: it initializes the workspace, transitions between stages, specifies each session’s objective and required deliverables, exposes the tool and resource interfaces, records and ranks scored solutions, persists run and session state, and enforces time and cost budgets. Within these boundaries, CLI-agent sessions freely decide their own research strategies, experiment plans, implementation details, and refinement processes.

Prepare Stage. Before iteration begins, EUREKAGENT launches a preparation agent session to set up a reliable runtime setup for subsequent solution iteration. The agent reads the problem description, the evaluator-facing submission requirements document, and optional initial code; tests

the hidden evaluation service; and installs or validates required runtime dependencies. If the problem setup is ambiguous or broken, the agent can pause and request human clarification rather than allowing optimization to proceed from an unreliable setup. The stage ends by writing a preparation summary and a completion artifact, which become shared context for later proposal and implementation sessions. This stage is executed only once at the beginning of the research process; after that, EUREKAGENT executes R rounds of propose–implement iteration.

Propose Stage. At the beginning of each iteration round, EUREKAGENT launches a proposal agent session to generate diverse initial hypotheses for the next round of solution optimization. The session reads the task inputs, the preparation summary, and the ranked best solutions from previous rounds, if any. It may also inspect previous-round workspaces for implementation details and use web search or browsing tools to gather related literature or existing open-source solutions. It then writes a manifest containing up to P candidate hypotheses and creates an implementation-ready description for each hypothesis. This stage acts as the fan-in step of EUREKAGENT: empirical evidence from earlier rounds, together with information from the internet, is distilled into a new set of promising, diverse, and independently executable research hypotheses.

Implement Stage. The implement stage is the fan-out step of EUREKAGENT. For each proposed hypothesis, EUREKAGENT launches a separate implementation agent session in parallel and assigns it a separate workspace. Each session starts from its assigned hypothesis as an initial direction, but may iteratively refine, debug, or modify the solution according to feedback from the hidden evaluator. Sessions submit candidate solutions through the secure evaluation service, which records all evaluated submissions and maintains both intermediate results and the best valid result. After the parallel implement sessions complete or exhaust their budgets, EUREKAGENT automatically ranks all valid submissions, and updates a ranked solution history file as shared context for the next round. This propose–implement loop combines broad parallel exploration with cross-round accumulation of empirical progress, continuing for improvement until the budget limits or stage completion conditions are reached.

3.2 ENVIRONMENT ENGINEERING IN EUREKAGENT

EUREKAGENT is designed through four environment engineering dimensions: (1) permissions engineering, (2) artifact engineering, (3) budget engineering, and (4) human-in-the-loop engineering. The goal is to grant agent sessions enough affordances to perform open-ended solution optimization, while making the research process reliable, inspectable, and resource-bounded.

Permissions Engineering. Scientific discovery agents need broad capabilities, but unconstrained capability can compromise research integrity. EUREKAGENT imposes system-level permission boundaries to support productive exploration while preventing research-integrity violations. On the productive side, EUREKAGENT provides a freely configurable Python environment, workspace-level shell access, capable web search and browser tools, and full access to the same run’s previous-round artifacts. This gives agents researcher-like access to tools, files, internet, and prior experience to aid solution iteration. On the constraint side, EUREKAGENT uses run-level isolation and controller-owned interfaces to prevent common failure modes. Each run executes inside a Docker container with a mounted workspace, protecting files outside the run from accidental or adversarial modification. The hidden evaluator with possible test data are kept outside the agent-visible workspace and exposed only through a secure grading service: agents can submit candidates and receive official scores, but cannot inspect or modify the evaluator itself. The authoritative result files generated by the hidden evaluator are automatically updated by the system, and hooks are implemented to block agent modification of these controller-owned files. EUREKAGENT also enforces same-round isolation among parallel implementation sessions: an implementation session may learn from previous rounds, but cannot inspect or copy from peer approaches in the same round, reducing premature collapse toward a single local direction. For GPU tasks, EUREKAGENT uses a default-deny policy: GPUs are invisible unless acquired through a provided GPU helper API, which records lock ownership and ensures that each physical GPU is held by at most one agent session at a time. Together, these mechanisms expose useful resources while removing high-risk affordances such as evaluator leakage, score tampering, uncontrolled GPU contention, and same-round solution copying.

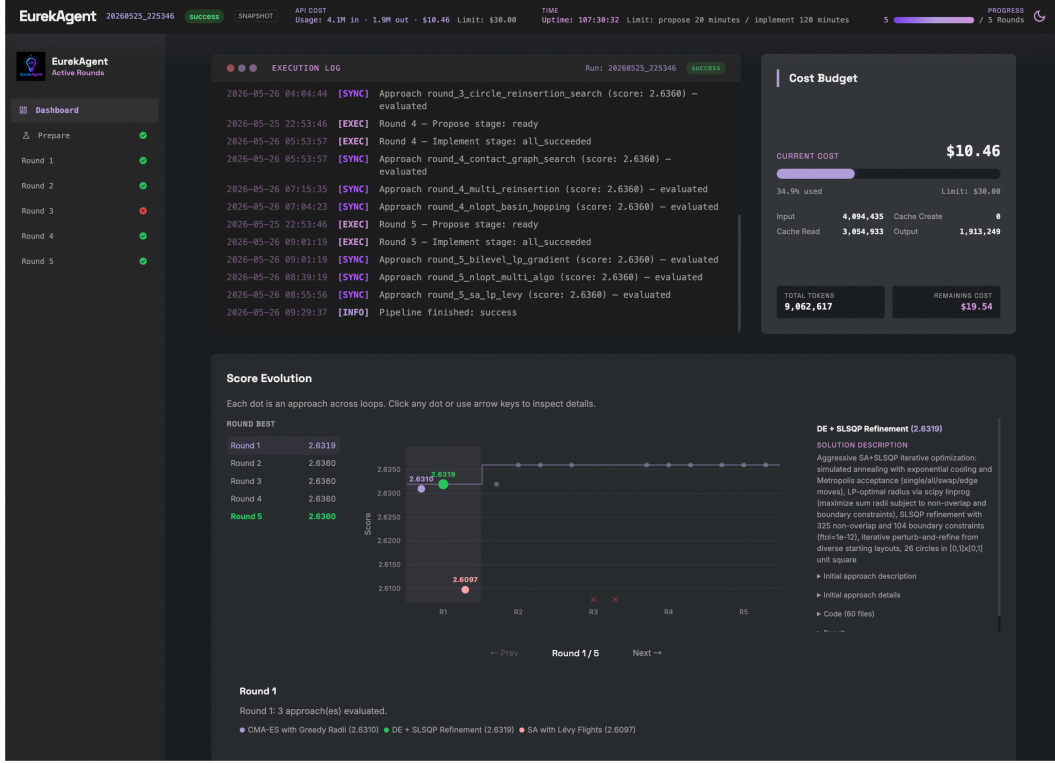


Figure 3: The EUREKAGENT web monitor interface. The monitor provides a user-friendly overview of each run, including status logs, score evolution, per-round and global best approaches, and budget usage. It also records complete session transcripts in an organized view, allowing users to inspect full trajectories of every agent session.

Artifact Engineering. EUREKAGENT uses the filesystem coupled with Git history as shared long-term memory. The filesystem stores stage deliverables for cross-session communication, including preparation summaries, proposal manifests, hypotheses, solution code, evaluator feedback, and scored submissions. EUREKAGENT also maintains system-managed artifacts: web-search history is logged as a cache of explored internet information, and official scores are automatically recorded and ranked. The ranked historical solutions enable later agent sessions to quickly identify strong prior solutions and inspect their code, logs, and intermediate results when needed. All run artifacts are persisted under the run directory, providing the persistent substrate for traceability, interruption recovery, and resumability. Within each session, Git commits track solution evolution. We instruct agents to describe both the current standalone solution and what changed from the previous version in each commit message.

Budget Engineering. Autonomous research agents can consume substantial time, compute, and API budget, so EUREKAGENT makes budget limits part of the environment settings. EUREKAGENT controls resources along two axes: wall-clock time and API cost. For time, users specify separate limits for proposal and implementation sessions, reflecting that hypothesis generation and long-running solution iteration require different time scales. Furthermore, EUREKAGENT makes agents time-aware through both active and passive mechanisms: (i) actively, agents can call a provided time-checking helper API to inspect elapsed and remaining time for the current stage; (ii) passively, when the deadline for a stage is approaching and required deliverables are still missing, EUREKAGENT injects a warning message asking the agent to stop exploration and generate the necessary artifacts. For API cost, EUREKAGENT tracks accumulated token usage across sessions, but does not expose token consumption information to the agent. When the cost limit is reached, the run is aborted and the current workspace is preserved as the final snapshot. Budget control also supports operational continuity for long-running research processes. EUREKAGENT persists each stage’s session identifier, status, elapsed time, and effective budget, so an interrupted run can resume


```

..EurekAgent..
AI-for-Research auto-experiment loop
glm-5.1 - 29.6K in - 361 out - $0.84
Prepare - validating setup - 0m35s - Monitor: 56791
/ Commands Ctrl+S Select Ctrl+C q Quit

Let me now read the initial code and evaluator, and set up the progress file.
Good, I have the problem and submission format. Let me now read the initial code and
evaluator, and create the progress file.

Tools: Bash
ls inputs/initial_code/

Tools: Bash
mkdir -p prepare

Tools: Read
/workspace/inputs/initial_code/initial.py

Tools: Bash
ls /workspace/eval/

Tools: Read
/workspace/eval/eureka_submit.py

Tools: Write
/workspace/prepare/progress.json

Type / for commands...

..EurekAgent..
AI-for-Research auto-experiment loop
glm-5.1 - 106.3K in - 36.5K out - $0.26
Round 1 of 5 - Implement - Monitor: 56791
ri Navigate - Enter session Ctrl+C q Quit
round_1_multi_start_nlp 10m/120m running 43.8K in 6.8K out
round_1_de_slsqp_hybrid 10m/120m running 27.1K in 16.9K out
round_1_greedy_physics 10m/120m running 14.5K in 5.9K out
Preview (round_1_greedy_physics):
This might be slow. Let me test it first and see.
Also, the greedy placement with 80x80 grid means checking 6400 points * 26 circles =
that's a lot of iterations. Let me estimate: for each of 26 circles, we check 6400
points, and for each point we compute distances to already-placed circles. That's roughly
26 * 6400 * 26/2 = ~2.1M distance computations. That should be manageable.
The physics refinement with 10000 iterations and 26*25/2 = 325 pairs per iteration =
3.25M pair computations. Should be OK.
The SLSQP might be the bottleneck. Let me try it.
Actually, let me commit and run it, and see what happens.
Tools: Bash
git add -A && git commit -m "Greedy sequential placement with physics refinement and
SLSQP optimization | Changed: initial implementation"
Tools: Bash
cd /workspace && python3 -u approach_details/round_1_greedy_physics/code/main.py 2>&1 |
tee approach_details/round_1_greedy_physics/logs/agent_run.log

```

Figure 4: The EUREKAGENT terminal UI interface. The terminal UI preserves a CLI-agent style view for inspecting live session outputs and communicating with agents through the bottom input box. Left: prepare-stage snapshot. Right: implement-stage snapshot with three parallel implementation sessions; users can preview all sessions or enter any session to inspect details and communicate.

from the latest filesystem state under the remaining budget rather than restarting from scratch. Users may also revise the configured time limits, or grant explicit extra resume time when a stage has exhausted its budget before producing required artifacts. This makes budget engineering not only a stopping rule, but also an operational interface for controlled continuation.

Human-in-the-loop Engineering. Although EUREKAGENT supports fully autonomous iteration, scientific discovery still benefits from human oversight. EUREKAGENT therefore provides two complementary interfaces. On one hand, a terminal UI (Figure 4) exposes per-approach progress, raw session outputs, and an input box for users to communicate with active sessions. On the other hand, a web monitor (Figure 3) provides higher-level views of the run, presenting a visualized score evolution with per-round and global best approaches. These interfaces preserve agent autonomy while keeping the process fully observable and allowing humans to redirect agent behavior when needed.

4 EXPERIMENTS

We evaluate EUREKAGENT on three domains: mathematics, kernel engineering, and machine learning engineering. We focus on tasks with optimizable metrics, where progress can be measured by objective scores. All experiments use EUREKAGENT with Claude Code as the CLI agent and GLM-5.1 as the base LLM. Building on this setting, we configure the Web Search Prime MCP² to enable search engine capabilities, and the Playwright MCP³ to enable web browser navigation.

4.1 MATHEMATICS

We evaluate EUREKAGENT on three mathematical optimization problems following prior work (Novikov et al., 2025; Wang et al., 2025; Yuksekogonul et al., 2026): (1) circle packing, where the objective is to place 26 non-overlapping circles inside a unit square and maximize the sum of their radii, using the OpenEvolve-style evaluator with a 10^{-6} tolerance for boundary and overlap checks; (2) Erdős’ minimum overlap, where the objective is to minimize the limiting maximum overlap between two equal-size sets; and (3) the first autocorrelation inequality, where the objective is to find a nonnegative construction that certifies the tightest known upper bound on an autoconvolution constant. For circle packing, we compare against the previous best AI result reported under the same tolerance setting. All three problems have verifiable and optimizable objective functions, making them suitable for agentic solution iteration. We report EUREKAGENT hyperparameter settings in Appendix A.

²<https://docs.z.ai/devpack/mcp/search-mcp-server>

³<https://github.com/microsoft/playwright-mcp>

Task	EUREKAGENT	LLM	Prev. Best AI	LLM
Circle Packing ($\uparrow$)	2.635 999	GLM-5.1	2.635986[27]	R1-Distill-Qwen3-8B
Erdős' Min. Overlap ($\downarrow$)	0.3808 70	GLM-5.1	0.380876[30]	gpt-oss-120b
1st Autocorr. Ineq. ($\downarrow$)	1.5028 61	GLM-5.1	1.502863[30]	gpt-oss-120b

Table 2: Performance of EUREKAGENT on three mathematical optimization problems. Previous best AI results are from test-time training systems, while EUREKAGENT remains training-free with only environment engineering designs.

As shown in Table 2, EUREKAGENT establishes new state-of-the-art results on all three mathematics tasks. Notably, EUREKAGENT outperforms prior test-time training systems while remaining training-free, suggesting that environment engineering alone can unlock breakthrough results without updating the backbone model.

4.2 KERNEL ENGINEERING

We evaluate EUREKAGENT on the GPUMODE TriMul competition, which targets optimized implementations of triangular matrix multiplication. Submissions are evaluated by the geometric mean runtime across benchmark cases, where lower runtime is better. We evaluate on the A100 setting.

Because the official GPUMODE leaderboard closed, we could not submit new solutions and get official scores. We therefore evaluate locally on an A100 GPU using the released TTT-Discover TriMul setting (Yuksekgonul et al., 2026), with only minimal format adaptation for EUREKAGENT submissions. For fair comparison, we download top leaderboard solution scripts from GPUMODE and regrade them under the same local protocol. All candidates are evaluated on the same A100 GPU with the original correctness tests, benchmark cases, scoring rule, and timing logic unchanged. We run three warmup rounds, followed by ten measured rounds with randomly shuffled candidate order to reduce order effects, and report both median and mean geometric-mean runtimes. For EUREKAGENT, we report the four best solutions discovered throughout a single system run; hyperparameter settings are listed in Appendix A.

Rank	Solution	LLM	Median (μ s) ($\downarrow$)	Mean (μ s) ($\downarrow$)
1	EUREKAGENT-CUDA Graph	GLM-5.1	2005.0307	2014.1874
2	EUREKAGENT-INT8 BMM	GLM-5.1	2006.9998	2013.5141
3	EUREKAGENT-Fused Front-End	GLM-5.1	2016.5718	2020.2674
4	EUREKAGENT-Triton Autotune	GLM-5.1	2030.6877	2041.5578
5	josusamartin	N/A	2096.0441	2105.1655
6	TTT-Discover[30]	gpt-oss-120b	2247.7849	2248.2307
7	rd9000	N/A	2300.4883	2307.5716

Table 3: Performance of EUREKAGENT on the TriMul kernel engineering task.

Table 3 shows that EUREKAGENT discovers multiple solutions that outperform the top leaderboard submissions under the same local evaluator. The top four EUREKAGENT solutions all achieve median runtimes below 2031 μ s, indicating stable high-quality optimization rather than a single lucky candidate. The best EUREKAGENT kernel improves over the strongest regraded leaderboard solution by about 4.3% and over TTT-Discover by about 10.8%.

4.3 MACHINE LEARNING ENGINEERING

We evaluate EUREKAGENT on a curated subset of seven competitions from the MLE-Bench Lite split (Chan et al., 2025). MLE-Bench evaluates agents on real Kaggle-style machine learning competitions, where submissions are scored against held-out test sets and mapped to medal thresholds. To balance cost, diversity, and difficulty, we start from the 22 Lite competitions and use public MLE-Bench leaderboard results to estimate tractability. We divide tasks into Easy, Medium, and Hard tiers by aggregate prior-agent medal rate, then sample 2 Easy, 2 Medium, and 3 Hard competitions. The

selected competitions span image, text, audio, and tabular prediction. Our selected tasks are detailed in Appendix B.

We run EUREKAGENT once per competition and report the resulting medal rates. Following MLE-Bench’s official 24-hour and single-GPU setting, we grant one GPU to each run and report EUREKAGENT hyperparameter settings in Appendix A. For baselines, we use the corresponding public MLE-Bench leaderboard results on the same tasks. When a baseline reports the aggregated scores of multiple runs, we report the upper end of its reported score range.

Rank	Agent	LLM	Any Medal	Gold	Above Median
1	EUREKAGENT	GLM-5.1	85.71%	71.43%	100.00%
2	AIBuildAI[31]	Claude-Opus-4.6	<u>71.43%</u>	57.14%	85.71%
3	Famou-Agent[12]	Gemini-2.5-Pro	<u>71.43%</u>	57.14%	100.00%
4	Famou-Agent 2.0[12]	Gemini-2.5-Pro	<u>71.43%</u>	65.39%	95.24%
5	LoongFlow[26]	Gemini-3-Flash-Preview	<u>71.43%</u>	57.14%	71.43%
6	CAIR MARS+[4]	Gemini-3-Pro-Preview	<u>71.43%</u>	71.43%	100.00%

Table 4: Machine learning engineering results on our seven-task MLE-Bench Lite subset. For baselines with multiple runs, we report the upper end of the reported range.

As shown in Table 4, EUREKAGENT achieves the highest any-medal rate on the selected MLE-Bench subset, reaching 85.71% with a single run per task. It also attains the highest gold-medal rate among methods using non-commercial open models. All listed baselines use closed commercial models, while EUREKAGENT runs with open-source LLM GLM-5.1, suggesting that environment-engineered autonomous iteration can be competitive even without relying on the strongest proprietary models.

5 CONCLUSION AND LIMITATIONS

We presented EUREKAGENT, an environment-engineered system for autonomous scientific discovery on metric-driven research tasks. Rather than prescribing detailed research workflows, EUREKAGENT coordinates off-the-shelf CLI-agent sessions through a simple prepare-propose-implement loop, while shaping the surrounding environment for reliable evaluation, shared progress memory, resource boundaries, and human oversight. Using Claude Code as the CLI agent and GLM-5.1 as the base LLM, EUREKAGENT achieves new state-of-the-art results on all evaluated mathematics and kernel engineering tasks, and ranks first on our evaluated MLE-Bench Lite subset. These results suggest that, as general-purpose CLI agents become more capable, carefully engineered scientific-discovery environments can turn model capability into reliable scientific progress.

Looking forward, we view environment engineering as a central layer in the next generation of autonomous research systems. As agents become more capable, scientific progress will depend not only on model intelligence, but also on the environments that define reliable feedback, persistent memory, resource control, evaluator integrity, human oversight, and recoverable long-running operation. Our current experiments focus on metric-driven tasks with executable evaluators, but the same perspective becomes even more important as autonomous research moves toward broader and more open-ended scientific settings.

We open-source EUREKAGENT as an initial step toward this direction and invite the community to build on, improve, and contribute to it. We will continue maintaining the project, extending it to richer research domains, and updating empirical results on its performance, capabilities, and boundaries. We hope EUREKAGENT can serve as a practical starting point for collective exploration of environment engineering as a foundation for reliable autonomous scientific discovery.

REFERENCES

- Anthropic. Claude Opus 4.7 system card. <https://www.anthropic.com/system-cards>, 2026.
- Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, et al. Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pp. 50466–50494, 2025.
- Jiefeng Chen, Bhavana Dalvi Mishra, Jaehyun Nam, Rui Meng, Tomas Pfister, and Jinsung Yoon. Mars: Modular agent with reflective search for automated ai research. *arXiv preprint arXiv:2602.02660*, 2026.
- Erich Friedman. Erich’s Packing Center. <https://erich-friedman.github.io/packing/>, 2024.
- James J. Gibson. *The Ecological Approach to Visual Perception*. Houghton Mifflin, Boston, MA, 1979.
- Jan Kristian Haugland. The minimum overlap problem revisited. *arXiv preprint arXiv:1609.08000*, 2016.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026.
- Natalia Kokoromyti. Anatomy of a reward hack: A real story from the latest GPU Mode NVFP4 competition. <https://www.gpumode.com/news/reward-hacking-nvfp4>, 2026.
- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- Annan Li, Chufan Wu, Zengle Ge, Yee Hin Chong, Zhinan Hou, Lizhe Cao, Cheng Ju, Jianmin Wu, Huaiming Li, Haobo Zhang, Shenghao Feng, Mo Zhao, Fengzhi Qiu, Rui Yang, Mengmeng Zhang, Wenyi Zhu, Yingying Sun, Quan Sun, Shunhao Yan, Danyu Liu, Dawei Yin, and Dou Shen. The FM agent, 2025. URL <https://arxiv.org/abs/2510.26144>.
- Jiaqi Liu, Shi Qiu, Mairui Li, Bingzhou Li, Haonian Ji, Siwei Han, Xinyu Ye, Peng Xia, Zihan Dong, Congyu Zhang, et al. Autoresearchclaw: Self-reinforcing autonomous research with human-ai collaboration. *arXiv preprint arXiv:2605.20025*, 2026a.
- Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z Pan, et al. Evox: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026b.
- Tengxiao Liu, Yuqing Yang, Xi Ye, and Danqi Chen. Can coding agents optimize algorithms autonomously? <https://tengxiaoliu.github.io/autoevolver/>, 2026c.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Ziming Luo, Atoosa Kasirzadeh, and Nihar B Shah. The more you automate, the less you see: Hidden pitfalls of ai scientist systems. *arXiv preprint arXiv:2509.08713*, 2025.
- Máté Matolcsi and Carlos Vinuesa. Improved bounds on the supremum of autoconvolutions. *Journal of mathematical analysis and applications*, 372(2):439–447, 2010.

- Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Arik, and Tomas Pfister. Mle-star: Machine learning engineering agent via search and targeted refinement. *Advances in Neural Information Processing Systems*, 38:116692–116712, 2026.
- Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher R  , and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.
- Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. Agentif: Benchmarking instruction following of large language models in agentic scenarios. *arXiv preprint arXiv:2505.16944*, 2025.
- Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, et al. Coral: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint arXiv:2604.01658*, 2026.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- Chunhui Wan, Xunan Dai, Zhuo Wang, Minglei Li, Yanpeng Wang, Yinan Mao, Yu Lan, and Zhiwen Xiao. Loongflow: Directed evolutionary search via a cognitive plan-execute-summarize paradigm. *arXiv preprint arXiv:2512.24077*, 2025.
- Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, et al. Thetaevolve: Test-time learning on open problems. *arXiv preprint arXiv:2511.23473*, 2025.
- Wanghan Xu, Shuo Li, Tianlin Ye, Qinglong Cao, Yixin Chen, Hengjian Gao, Yiheng Wang, Qi Li, Kun Li, Sheng Xu, Shengdu Chai, Fangchen Yu, Xiangyu Zhao, Zhangrui Zhao, Weijie Ma, Zijie Guo, Haoyu Zhou, Haoxiang Yin, Lixue Cheng, Chaofan Hu, Haoxuan Li, Lu Mi, Xuxuan Xie, Yifan Zhou, Ruizhe Chen, Zhiwang Zhou, Xingjian Guo, Yuhao Zhou, Xuming He, Shengyuan Xu, Xinyu Gu, Jiamin Wu, Mianxin Liu, Chunfeng Song, Fenghua Ling, Dongzhan Zhou, Shixiang Tang, Yuqiang Li, Mao Su, Peng Ye, Siqi Sun, Bin Wang, Xue Yang, Zhenfei Yin, Tianfan Fu, Guangtao Zhai, Wanli Ouyang, Bo Zhang, Lei Bai, and Wenlong Zhang. Researchclawbench: A benchmark for end-to-end autonomous scientific research, 2026. URL <https://arxiv.org/abs/2606.07591>.
- Xu Yang, Xiao Yang, Shikai Fang, Bowen Xian, Yuante Li, Jian Wang, Minrui Xu, Haoran Pan, Xinpeng Hong, Weiqing Liu, et al. R&d-agent: Automating data-driven ai solution building through llm-powered automated research, development, and evolution. *arXiv e-prints*, pp. arXiv–2505, 2025.
- Mert Yuksekgonul, Daniel Kocaja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.
- Ruiyi Zhang, Peijia Qin, Qi Cao, Li Zhang, and Pengtao Xie. Aibuildai: An ai agent for automatically building ai models. *arXiv preprint arXiv:2604.14455*, 2026.
- Xinyu Zhu, Yuzhu Cai, Zexi Liu, Bingyang Zheng, Cheng Wang, Rui Ye, Jiaao Chen, Hanrui Wang, Wei-Chen Wang, Yuzhi Zhang, et al. Toward ultra-long-horizon agentic science: Cognitive accumulation for machine learning engineering. *arXiv preprint arXiv:2601.10402*, 2026.

A EUREKAGENT HYPERPARAMETER SETTINGS

Table 5 summarizes the EUREKAGENT hyperparameters used in our experiments. Here, R denotes the maximum number of propose–implement iteration rounds, and P denotes the maximum number of parallel implementation sessions spawned in each implement stage.

Task	R	P	$t_{propose}$	$t_{implement}$	Notes
Circle Packing	5	3	20 min	120 min	–
Erdős’ Min. Overlap	8	3	20 min	120 min	–
1st Autocorr. Ineq.	8	3	20 min	120 min	–
TriMul	13	3	20 min	160 min	A100 evaluation setting.
MLE-Bench Lite	12	3	20 min	100 min	One GPU per run.

Table 5: EUREKAGENT hyperparameter settings used in our experiments.

B SELECTED MLE-BENCH LITE COMPETITIONS

We select seven MLE-Bench Lite competitions across three difficulty tiers, using aggregate medal rates of prior public leaderboard agents as a proxy for task difficulty:

- **Easy** ($> 40\%$): histopathologic-cancer-detection (57.0%) and plant-pathology-2020-fgvc7 (49.7%).
- **Medium** (15%–40%): aerial-cactus-identification (26.2%) and the-icml-2013-whale-redux (23.5%).
- **Hard** ($< 10\%$): jigsaw-toxic-comment (9.1%), dog-breed-identification (0.4%), and tabular-playground-may-2022 (0.4%).